



UNIVERSITÀ
DEGLI STUDI
DI PADOVA

Università degli Studi di Padova

Laurea in Informatica

Corso di Ingegneria del Software

Anno Accademico 2025/2026



Gruppo ByteMe

byteme2025swe@gmail.com

Glossario

Versione	1.2.6
Stato	Da Approvare
Redazione	Tutto il gruppo
Verifica	Tutto il gruppo
Approvazione	Tutto il gruppo
Proprietario	ByteMe
Uso	Interno

Registro dei cambiamenti

Versione	Data	Autore	Descrizione	Verifica
1.2.6	27/12/2025	Joseph Grant	Aggiunta sezione 'verifica' al versionamento	Elisa Marchioro
1.2.5	15/12/2025	Giulia Barzon	Aggiunte parole relative a Piano di qualifica e testing	Elisa Marchioro
1.2.4	14/12/2025	Giulia Barzon	Aggiunte parole requisiti funzionali, non funzionali, di qualità, di vincolo e Piano di qualifica	
1.2.3	8/12/2025	Elisa Marchioro	Aggiunte parole toolbar, user story, milestone, issue, commit, merge	
1.2.2	5/12/2025	Giulia Barzon	Verifica documento e aggiunte parole relative al Piano di Progetto e ai casi d'uso	
1.2.1	4/12/2025	Elisa Marchioro	Aggiunta parole agile, scrum, varie fasi dello sprint, RTB, PB, CA, verifica, validazione	
1.1.1	28/11/2025	Elisa Marchioro	Piccoli fix al documento	
1.1.0	26/11/2025	Elisa Marchioro	Aggiunta parole analisi requisiti, API, backlog, database, distant writing, efficienza, efficacia, git, github, LLM, markdown, MVP, PoC, repository, six thinking hats, sprint	
1.0.0	18/11/2025	Chiara Grossele	Stesura iniziale del documento	

Tabella 1: Registro delle versioni del documento

Indice

1	Introduzione	1
2	Struttura del documento	1
A		2
B		3
C		4
D		5
E		6
F		7
G		8
H		9
I		10
J		11
K		12
L		13
M		14
N		15
O		16
P		17
Q		18
R		19
S		20
T		21
U		22
V		23
W		24
X		25
Y		26
Z		27

1 Introduzione

Il presente documento ha lo scopo di raccogliere i termini utilizzati nella documentazione del gruppo ByteMe per il progetto di Ingegneria del Software nell'anno accademico 2025/2026. L'intento dietro a questo documento è la delucidazione di termini sconosciuti e specializzati del dominio del progetto.

2 Struttura del documento

Il documento è strutturato in sezioni dedicate ad una specifica lettera dell'alfabeto. I termini contenuti in questo documento sono stati appresi dalle lezioni, dalle diapositive del corso di Ingegneria del Software e da uno studio autonomo del gruppo.

A

- **Affidabilità (Reliability):** È la capacità del prodotto software di svolgere le proprie funzioni specificate sotto determinate condizioni e per un determinato periodo di tempo, mantenendo un livello di prestazioni corretto. Metrica tipica: Failure Density (densità di fallimenti nei test).
- **Agile:** È un modo di sviluppare software basato su cicli di lavoro brevi, collaborazione continua e adattamento ai cambiamenti, così da poter migliorare il prodotto passo dopo passo e consegnare rapidamente valore all'utente.
- **Analisi dei requisiti:** Fase in cui si raccolgono e si comprendono i bisogni del cliente e cosa deve fare il software.
- **Analisi Dinamica:** Attività di verifica che prevede l'esecuzione del codice software (oggetti di prova) su un processore reale o virtuale, fornendo dati in input e controllando che l'output corrisponda ai risultati attesi (oracolo). Corrisponde ai test "veri e propri" (Test di unità, Test di integrazione, Test di sistema, Test di accettazione, Test di Regression).
- **Analisi Statica:** Attività di controllo del software effettuata senza eseguire il codice. Include la revisione dei documenti, l'analisi automatica della sintassi e del rispetto degli standard di codifica tramite strumenti. Appartiene alla categoria dei test non funzionali e aiuta a prevenire difetti prima dell'esecuzione.
- **API:** Insieme di regole che permette a diversi software di comunicare tra loro. È come un "ponte" che permette a un software di usare funzioni o dati di un altro software.

B

- **Backlog:** Elenco delle cose da fare nel progetto, organizzato in base a priorità.
- **Branch Coverage (Copertura dei Rami):** Metrica specifica di copertura che verifica se ogni singolo ramo (decisione vero/falso) del flusso di controllo è stato attraversato almeno una volta dai test.

C

- **CA (Costumer Acceptance):** Fase in cui il cliente controlla il prodotto consegnato e conferma che rispetta i requisiti richiesti.
- **Checklist:** Lista di controllo contenente elementi specifici da verificare manualmente durante le attività di revisione (Inspection o Walkthrough) di documenti o codice. Serve a garantire che errori comuni (es. ortografia, formattazione, mancanza di caption) siano individuati e corretti in modo sistematico.
- **Code Coverage (Copertura del Codice):** Misura che indica la percentuale di codice sorgente che viene eseguita durante l'esecuzione di una suite di test automatici. Serve a valutare l'efficacia dei test: più è alta, minore è il rischio di difetti non rilevati, anche se il cento per cento non garantisce l'assenza di errori.
- **Commit:** Registrazione di un insieme di modifiche nella repository, accompagnata da un messaggio che descrive cosa è stato modificato.
- **Complessità Ciclomantica:** Metrica software che misura la complessità del flusso di controllo di un programma contandone il numero di percorsi linearmente indipendenti. Si calcola con la formula $v(G) = e - n + 2p$ (dove e sono gli archi, n i nodi, p le componenti connesse). Un valore alto indica un codice difficile da testare e mantenere, poiché cresce con la presenza di branch, salti e iterazioni.
- **Cruscotto di Qualità:** sezione del Piano di Qualifica (o uno strumento software) che visualizza in modo sintetico lo stato corrente del progetto rispetto agli obiettivi prefissati. Utilizza grafici e tabelle per confrontare i valori misurati (es. costi, tempi, errori) con i valori attesi (ottimali o accettabili). Serve a monitorare l'andamento nel tempo di metriche definite nel Piano di Qualifica.

D

- **Database:** Sistema per memorizzare e gestire dati in modo organizzato.
- **Debito Tecnico (Technical Debt):** metafora che indica il costo implicito di una rielaborazione aggiuntiva causata dalla scelta di una soluzione facile e rapida (ma non ottimale) invece di utilizzare un approccio migliore che richiederebbe più tempo. Può essere consapevole (deliberato) o inconsapevole (inavvertito). Se non si "ripaga" il debito (es. facendo refactoring), il costo degli errori residui e della manutenzione cresce nel tempo.
- **Distant Writing:** Modalità di scrittura in cui l'utente fornisce le specifiche e l'LLM genera il testo completo.

E

- **Editor di Testo:** Componente principale dell'interfaccia utente del sistema "Second Brain". Si tratta di un editor online avanzato che supporta la sintassi Markdown, fornendo all'utente un'area di scrittura con anteprima in tempo reale del testo formattato. È l'ambiente in cui l'utente crea, modifica e visualizza i contenuti, nonché il punto di interazione per invocare le funzionalità del Sistema AI sul testo selezionato o completo.
- **Efficacia:** Misura della capacità di raggiungere l'obiettivo prefissato.
- **Efficienza (Efficiency):** Misura dell'abilità di raggiungere l'obiettivo impiegando le risorse minime indispensabili. È la capacità del prodotto software di fornire prestazioni adeguate (es. tempo di risposta) in relazione alla quantità di risorse utilizzate, sotto condizioni stabilite. Metrica tipica: Tempo di risposta medio.

F

G

- **Git:** Sistema di controllo versione che permette di tenere traccia delle modifiche al codice sorgente e di collaborare in modo efficiente.
- **Github:** Piattaforma online basata su Git che consente di ospitare repository, collaborare al progetto e gestire versioni e richieste di modifica.
- **Glossario:** Raccolta di vocaboli meno comuni in quanto limitati a un ambiente o propri di una determinata disciplina, accompagnati ognuno dalla spiegazione del significato o da altre osservazioni. Nel nostro caso raccoglierà tutte le parole relative al nostro progetto che meritano di ulteriori precisazioni o che vengono applicate in un particolare contesto.

H

I

- **Inspection (Ispezione):** Tecnica di revisione statica formale e strutturata, eseguita da un gruppo di pari con l'obiettivo specifico di rilevare difetti e non conformità. A differenza del Walkthrough, l'Inspection si basa sull'uso di checklist per verificare il rispetto di standard e regole predefinite. Nel Piano di Qualifica l'Inspection è spesso preferita al Walkthrough per le attività di verifica ufficiale perché garantisce un controllo più oggettivo e approfondito grazie alle checklist.
- **Issue:** Attività, richiesta o problema tracciato all'interno di un sistema di gestione del progetto (come GitHub), che richiede attenzione o sviluppo.

J

K

L

- **LLM (Large Language Model):** Modello di intelligenza artificiale addestrato sulla comprensione e generazione di linguaggio naturale.

M

- **Manutenibilità (Maintainability):** È la facilità con cui il prodotto software può essere modificato da parte dei manutentori per correggere difetti, adattarlo a nuovi ambienti o soddisfare nuovi requisiti. Metrica tipica: Complessità Ciclomatica (più è bassa, più è manutenibile).
- **Markdown:** Linguaggio di markup leggero per la formattazione di testo semplice.
- **MVP (Minimum Viable Product):** Versione minima del prodotto contenente tutte le funzionalità obbligatorie.
- **Milestone:** Punto significativo nel percorso del progetto che segna il completamento di una fase o un risultato importante.
- **Merge:** Operazione che combina le modifiche provenienti da un ramo di sviluppo con un altro, integrando il lavoro dei vari membri del team.

N

- **Norme di Progetto:** Documento formale che definisce e descrive il Way of Working adottato dal gruppo. Contiene le regole, le convenzioni, i processi e le linee guida che il team si impegna a seguire per l'organizzazione del lavoro, la comunicazione, la gestione della repository, la redazione e verifica della documentazione, l'assegnazione dei ruoli e la pianificazione delle attività. Le Norme di Progetto costituiscono un riferimento condiviso per garantire coerenza, qualità e controllo durante lo svolgimento del progetto, e sono soggette a versionamento e aggiornamento in base all'evoluzione delle esigenze del team e del progetto stesso.

O

P

- **PB (Product Baseline)**: È la definizione completa del prodotto dal punto di vista funzionale e progettuale, usata come base per sviluppo, test e verifica. Include anche l'MVP (Minimum Viable Product), cioè la versione minima del prodotto che contiene solo le funzionalità essenziali per essere testata dagli utenti e ricevere feedback.
- **PDCA (Plan-Do-Check-Act)**: metodo di gestione iterativo utilizzato per il controllo e il miglioramento continuo dei processi e dei prodotti. Nel Piano di Qualifica rappresenta la strategia di gestione della qualità: Plan (pianificare obiettivi e metriche), Do (eseguire le attività), Check (verificare i risultati tramite il Cruscotto), Act (applicare correzioni se i risultati non sono accettabili).
- **Piano di Progetto (PdP)**: Documento strategico e operativo che definisce il piano di esecuzione complessivo del progetto, descrivendo come il team intende raggiungere gli obiettivi prefissati. Il suo scopo è fornire una roadmap strutturata e condivisa, che indichi cosa deve essere fatto, da chi, con quali risorse, entro quando e quali rischi si possono incontrare. Serve come base per organizzare il lavoro, monitorare lo stato di avanzamento, gestire le deviazioni e comunicare in modo trasparente l'andamento a committenti e proponenti.
- **Piano di Qualifica (PdQ)**: Documento che definisce la strategia, le metodologie e le risorse necessarie per garantire la qualità del prodotto software e del processo di sviluppo. Esso stabilisce gli obiettivi di qualità (tramite metriche e soglie di accettazione), pianifica le attività di verifica e validazione (analisi statica e dinamica) e descrive le procedure di miglioramento continuo (ciclo PDCA). Il PdQ funge da riferimento per assicurare che il progetto soddisfi i requisiti funzionali, di qualità e di vincolo concordati.
- **Proof of Concept (PoC)**: Prototipo iniziale e limitato del software, sviluppato per dimostrare la fattibilità delle funzionalità core e dell'idea del progetto in generale.

Q

- **Quality Gates (Cancelli di Qualità):** Criteri di controllo automatico imposti al codice per verificare se la qualità rispetta gli standard minimi definiti. Permettono di stabilire se il progetto può passare a uno stadio successivo o se una modifica può essere accettata. Se il codice non supera il *Quality Gate*, la modifica viene bloccata.

R

- **Repository:** Spazio in cui viene salvato il codice del progetto insieme a tutte le versioni e i file utili. Permette al team di lavorare insieme senza perdere il lavoro svolto.
- **Requisiti funzionali:** Descrivono i comportamenti del sistema. Sono le azioni che il software deve compiere in risposta a determinati input. Se un requisito funzionale non è soddisfatto, manca una funzionalità. Rispondono alla domanda: *Cosa deve fare il software?*
- **Requisiti non funzionali:** I requisiti non funzionali descrivono le proprietà del sistema o i vincoli entro cui deve operare. Non aggiungono "cose da fare", ma stabiliscono quanto bene il sistema deve farle. Rispondono alla domanda: *Come si deve comportare il sistema mentre fa le cose?*
- **Requisiti di qualità:** I requisiti di qualità definiscono quanto bene il sistema deve funzionare. Non riguardano cosa fa il sistema, ma le sue caratteristiche intrinseche come affidabilità, manutenibilità, efficienza e usabilità. Spesso derivano da standard ISO (come ISO/IEC 25010). Rispondono alla domanda: *Quali standard di eccellenza deve rispettare il mio codice/prodotto?*
- **Requisiti di vincolo:** I vincoli sono limitazioni imposte dall'esterno (dal committente, dalla legge, o dalla tecnologia disponibile) entro le quali dobbiamo muoverci. Non abbiamo libertà di scelta su questi aspetti, sono i "paletti" del progetto. Esempio requisiti di vincolo:
 - Tecnologici: il progetto deve essere un'applicazione web basata su HTML/JS e che l'editor deve essere basato su Markdown.
 - Architetture: l'applicazione deve funzionare prevalentemente client-side (sul browser).
 - Interfaccia esterna: usare le API standard (tipo OpenAI) per collegarsi all'LLM.
 - Licenza/Consegna: il codice deve essere pubblicato su un repository pubblico (es. GitHub) con licenza open-source (non so se considerarlo un requisito).
- **RTB (Requirements and Technology Baseline):** Punto di riferimento formale e congelato del progetto, rappresentato da un documento approvato che stabilisce in modo definitivo l'insieme dei requisiti funzionali e non funzionali del sistema, insieme alle scelte architetture e tecnologiche fondamentali su cui si baserà lo sviluppo.

S

- **Scrum:** È un metodo Agile che organizza il lavoro in Sprint regolari e utilizza ruoli ed eventi definiti per aiutare il team a lavorare in modo ordinato e trasparente.
- **Sistema AI:** Componente software del sistema "Second Brain" che integra un Large Language Model (LLM) per fornire funzionalità di elaborazione intelligente del testo. È un attore secondario che opera in modo trasparente all'utente, abilitando operazioni come: riassunto, riscrittura, traduzione, correzione grammaticale, analisi critica tramite il metodo dei "Sei Cappelli per Pensare", e generazione di testo automatico (distant writing). Il sistema AI interagisce con il testo selezionato o completo inserito nell'editor.
- **Sistema di Persistenza:** Componente architetturale del sistema "Second Brain" responsabile della memorizzazione, del recupero e della gestione dei dati degli utenti. È un attore secondario che opera a livello locale, cioè gestisce il salvataggio e il caricamento di file direttamente sul dispositivo dell'utente; a livello cloud (opzionale per utenti autenticati), cioè gestisce un archivio centralizzato su database server-side, abilitando operazioni CRUD (Create, Read, Update, Delete) remote e accesso sincronizzato ai documenti.
- **Six Thinking Hats (Sei Cappelli per Pensare):** Metodologia di Edward De Bono per l'analisi critica da diverse prospettive.
- **Sprint:** Periodo di lavoro breve e programmato (2 settimane nel nostro caso) in cui il team realizza un insieme di attività. Durante lo sprint i membri del gruppo cambiano ruolo per favorire collaborazione e crescita delle competenze.
- **Sprint Planning:** Riunione che apre lo Sprint, in cui il team decide quali attività svolgere e come affrontarle.
- **Sprint Execution:** La fase dello Sprint in cui il team lavora concretamente sulle attività pianificate, sviluppando e testando le funzionalità.
- **Sprint Review:** Incontro alla fine dello Sprint dove il team mostra il lavoro completato e raccoglie feedback da Product Owner e stakeholder.
- **Sprint Retrospective:** Riunione conclusiva dello Sprint in cui il team riflette sul proprio modo di lavorare e propone miglioramenti per lo Sprint successivo.

T

- **Toolbar:** Elemento dell'interfaccia grafica che raccoglie pulsanti e funzioni di uso frequente, permettendo un accesso rapido alle operazioni principali.
- **Test di Accettazione (Acceptance Testing / Collaudo):** Validazione finale per accertare il soddisfacimento dei requisiti utente, svolta nella fase finale prima del rilascio.
- **Test di Integrazione:** Verificano le interfacce e l'interazione tra moduli o sottosistemi già testati singolarmente. Vengono fatti dopo i test di unità, risale il V-Model verso l'architettura (Global Design). Rilevano difetti nella progettazione architetturale o nello scambio dati tra componenti.
- **Test di Regression:** Non è una fase separata, ma una pratica trasversale. Ogni volta che si modifica il codice o si aggiunge una funzionalità, si eseguono di nuovo i test precedenti per assicurarsi di non aver rotto funzionalità già esistenti.
- **Test di Sistema:** Verificano il comportamento dell'intero sistema software completo rispetto alle specifiche tecniche. Verificano il sistema completo rispetto ai requisiti funzionali e non funzionali. Includono test non funzionali (performance, sicurezza, usabilità). Comprendono anche smoke test (test rapidi di stabilità). Vengono fatti dopo l'integrazione, corrisponde alla verifica dei "System Requirements".
- **Test di Unità (Unit Testing):** È la verifica del più piccolo componente software funzionante autonomamente (es. un metodo o una classe in Java). Vengono fatti durante la codifica (Coding) o addirittura prima (TDD - Test Driven Development). Corrispondono alla fase di "Detailed Design" nel V-Model. Devono essere Automatici, Thorough (accurati/esaustivi), Ripetibili, Indipendenti (l'ordine non conta) e Professionali (codice di qualità).

U

- **Usabilità (Usability):** È il grado con cui un prodotto può essere usato da specifici utenti per raggiungere specifici obiettivi con efficacia, efficienza e soddisfazione. Include la facilità di apprendimento e comprensione. Metrica tipica: Tempo di apprendimento, Numero di click per raggiungere un obiettivo.
- **User story:** Descrizione sintetica di una funzionalità dal punto di vista dell'utente, utilizzata nei metodi Agile per chiarire cosa il sistema deve permettere di fare.
- **Utente Autenticato:** Categoria di utente che ha completato il processo di registrazione e/o accesso al sistema. Questo profilo abilita l'accesso a funzionalità avanzate del prodotto, tra cui la persistenza cloud dei propri documenti, la gestione remota (CRUD) dei file e l'accesso persistente e sincronizzato alle note senza operazioni manuali di upload/download. È un attore primario nel sistema, destinato a utenti che necessitano di un ambiente di lavoro persistente e personalizzato.
- **Utente Non Autenticato:** Categoria di utente che interagisce con il sistema senza effettuare registrazione o login. Ha accesso alle funzionalità base del prodotto, tra cui l'utilizzo dell'editor Markdown, le operazioni AI su testo (riassunto, riscrittura, correzione, etc.) e la persistenza locale dei file. È un attore primario nel sistema, pensato per un utilizzo immediato, occasionale o privo dell'esigenza di salvataggio remoto e sincronizzazione.

V

- **Validazione:** Processo che verifica se il prodotto risponde ai bisogni reali dell'utente e quindi se è il prodotto giusto da realizzare.
- **Verifica:** Processo che controlla se il prodotto è stato realizzato correttamente, cioè secondo specifiche e progettazione.
- **Versionamento:** Pratica sistematica di assegnare identificativi univoci e incrementali alle diverse versioni di un documento o di un componente software, al fine di tracciarne l'evoluzione nel tempo; garantisce tracciabilità, consente il controllo delle modifiche e facilita il ripristino di versioni precedenti.
- **V-Model (Modello a V):** modello di sviluppo software sequenziale che evidenzia la relazione diretta "uno a uno" tra le fasi di sviluppo (lato sinistro della V) e le corrispondenti fasi di test (lato destro della V). Serve a capire che non si testa "alla fine", ma ogni fase di test verifica una specifica fase di progettazione.

W

- **Walkthrough:** Tecnica di revisione statica (generalmente informale) in cui l'autore di un artefatto software (come un documento di requisiti o una porzione di codice) guida i membri del team attraverso il suo contenuto per ricevere feedback, favorire la comprensione comune e individuare errori evidenti. È utile per la condivisione della conoscenza all'interno del team, ma è meno rigoroso dell'Inspection.
- **Way of Working:** Insieme strutturato di regole, processi, strumenti e pratiche adottate da un gruppo di lavoro per organizzare, coordinare e gestire le attività di un progetto. Definisce le modalità di comunicazione interna ed esterna, la gestione del codice e della documentazione, l'assegnazione e rotazione dei ruoli, nonché i flussi di verifica e approvazione. Il Way of Working ha lo scopo di garantire efficienza, tracciabilità, collaborazione e miglioramento continuo durante l'intero ciclo di vita del progetto.

X

Y

Z